

REF. NO.: EX/EE/PC/B/T/225/2025

SEQUENTIAL SYSTEMS & MICROPROCESSORS

B.E. Electrical Engineering · 2nd Year · 2nd Semester Examination, 2025

TIME: 3 HOURS FULL MARKS: 100 EACH PART: 50

PART I – SEQUENTIAL SYSTEMS

50 marks

Q1

Answer *any Two* (compulsory; $2 \times 5 = 10$ marks)

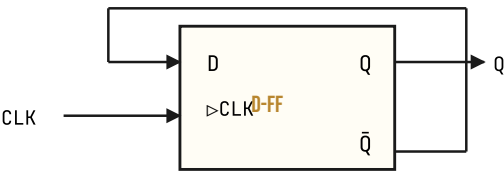
(A) D FLIP-FLOP IN TOGGLE MODE

5

The D FF has the simplest characteristic equation $Q^+ = D$. To make it toggle ($Q^+ = \bar{Q}$ on every clock) we must drive D with $\bar{Q}$:

$$D = \bar{Q} \Rightarrow Q^+ = \bar{Q}$$

This is achieved by feeding the FF's $\bar{Q}$ output back to its D input. The output then changes on every active clock edge, dividing the clock frequency by 2.



D FF wired as T FF: feeding back $\bar{Q}$ to D produces toggle behaviour.

CLOCK	Q (BEFORE)	D = $\bar{Q}$	Q (AFTER)
1	0	1	1
2	1	0	0
3	0	1	1

(B) EXCITATION TABLE FOR A FLIP-FLOP; JK EXCITATION TABLE

5

The **excitation table** of a flip-flop tabulates, for every possible state transition $Q \rightarrow Q^+$, the values that the FF's control inputs must take to *cause* that transition. It is the inverse of the truth/characteristic table — used during synthesis (when the desired next state is known and one needs to determine inputs).

JK EXCITATION TABLE

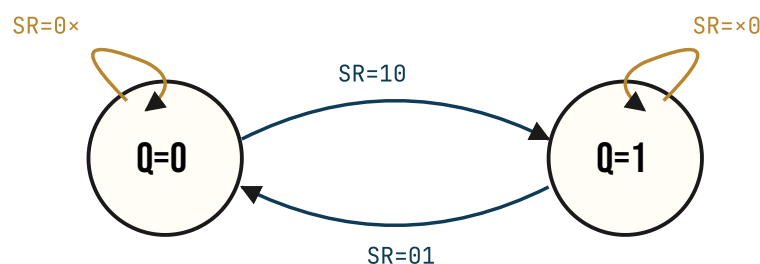
Q	Q ⁺	J	K	ACTION
0	0	0	x	Hold/Reset (J=0)
0	1	1	x	Set/Toggle (J=1)
1	0	x	1	Reset/Toggle (K=1)
1	1	x	0	Hold/Set (K=0)

Don't-cares (x) appear because two distinct (J,K) combinations can produce the same Q^+ — e.g. for $0 \rightarrow 0$ both 00 (hold) and 01 (reset) work. These don't-cares give freedom during K-map minimisation.

(C) STATE DIAGRAM OF S-R FF FROM ITS EXCITATION TABLE

5

Q	Q ⁺	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0



SR FF has two states; SR=11 is forbidden (excluded from the diagram).

(D) $J\bar{K}$ FF WITH $K = \bar{J}$ — SINGLE COMBINED INPUT — IDENTIFY IT

5

A " $J\bar{K}$ " FF is a JK FF with an inverter on K, so internally $K_{int} = \bar{K}_{ext}$. With a single external input X tied to both J and K :

- Feeding X to J : $J = X$.

- Feeding X to $\bar{K}$ input $\rightarrow K_{int} = \bar{X}$ (already inverted by the structural inverter; tying both pins to X means the original K-input is X , and the FF feeds K inverted internally, so net $K_{int} = \bar{X}$).

So the effective JK seen by the core: $(J, K) = (X, \bar{X})$.

X	J	K	Q^+
0	0	1	0 (Reset)
1	1	0	1 (Set)

Hence $Q^+ = X$. **X is the D Flip-Flop**: it loads the input directly on every clock pulse, behaving as a 1-bit register. Justification: $X = 0 \Rightarrow \text{reset}$, $X = 1 \Rightarrow \text{set}$ — exactly the D FF's characteristic equation $Q^+ = D$.

Q2

State Synthesis Table; SR $\rightarrow$ JK; analysis of Fig. Q2(b)

8 + 12

[A] STATE SYNTHESIS TABLE; SR $\rightarrow$ JK CONVERSION

2 + 6

The **State Synthesis Table** consolidates: present state Q , external inputs of the target FF (here J, K), the desired next state Q^+ obtained from the target's truth table, and the inputs of the source FF (here S, R) that effect that transition (read from the source's excitation table). It is the working sheet from which the FF's input expressions are derived by K-map.

Q	J	K	Q^+	S	R
0	0	0	0	0	x
0	0	1	0	0	x
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	x	0
1	0	1	0	0	1
1	1	0	1	x	0
1	1	1	0	0	1

K-map minimisation:

$$S = J\bar{Q}, \quad R = KQ$$

[B] ANALYSE THE SEQUENTIAL CIRCUIT OF FIG. Q2(B)

8 + 4

Reading the figure: two SR FFs A and B both clocked by CLK. The gate equations are:

$$S_A = \bar{B}, \quad R_A = B$$

$$S_B = A\bar{B}, \quad R_B = \bar{A}B$$

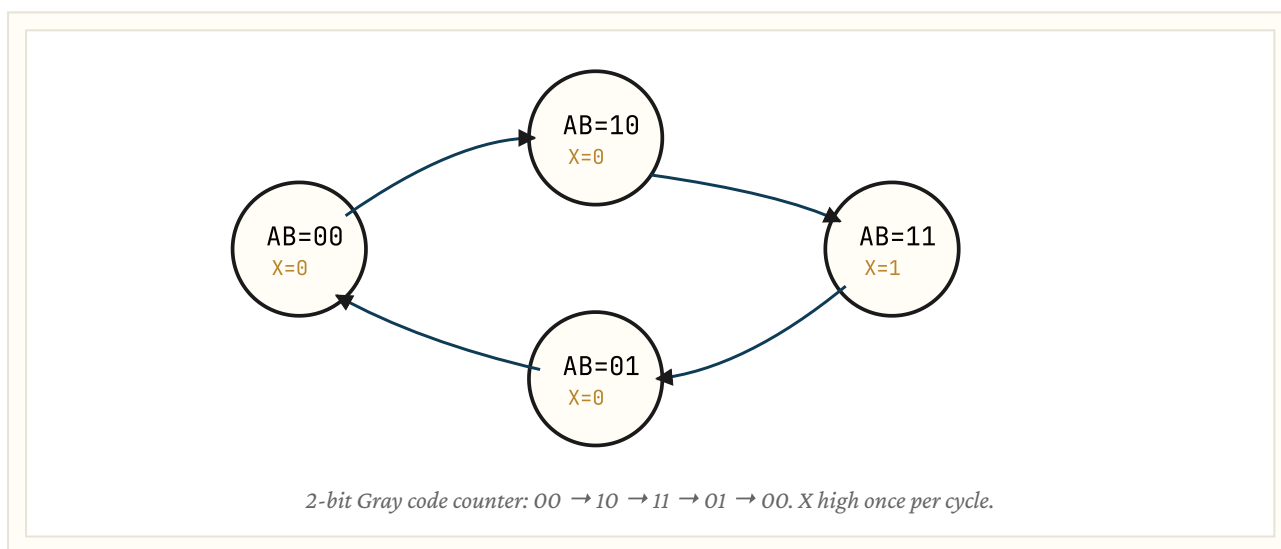
$$X = AB$$

STATE TABLE

A	B	$S_A R_A$	$S_B R_B$	A^+	B^+	X
0	0	10	00	1	0	0
0	1	01	01	0	0	0
1	0	10	10	1	1	0
1	1	01	00	0	1	1

Cycle: 00 $\rightarrow$ 10 $\rightarrow$ 11 $\rightarrow$ 01 $\rightarrow$ 00... — a 2-bit Gray-code counter; output X pulses high for one CLK in every four (when AB = 11).

STATE TRANSITION DIAGRAM



Q3

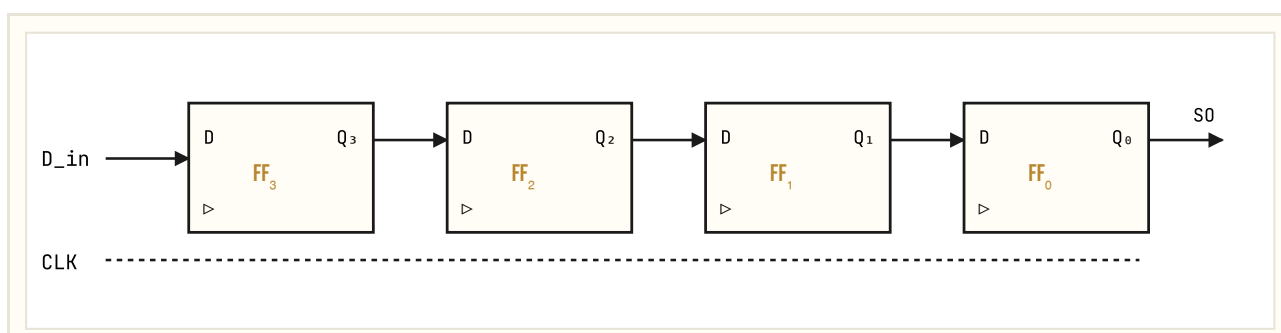
4-bit shift-right register • Hybrid 4-bit counter (Fig. Q3(b))

10 + 10

(A) 4-BIT SHIFT-RIGHT REGISTER; SERIAL ENTRY OF "0100"

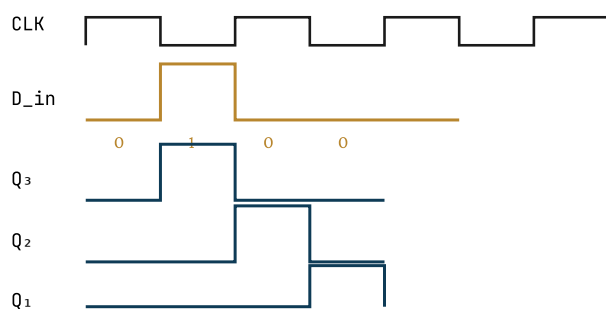
4 + 6

Cascade four D FFs sharing a common clock; the serial input drives FF3 and each FF feeds its right neighbour:



SERIAL ENTRY OF "0100" (LSB OF BINARY FIRST)

CLOCK	D_{IN}	Q_3	Q_2	Q_1	Q_0
Init	—	0	0	0	0
1	0	0	0	0	0
2	1	1	0	0	0
3	0	0	1	0	0
4	0	0	0	1	0



After 4 CLK pulses, the register holds 0010 $\leftrightarrow$ "0100" loaded serially.

(B) SYNCHRONOUS VS ASYNCHRONOUS; SHOW FIG. Q3(B) IS HYBRID

2 + 8

Synchronous counter: all FFs share a single clock and change states simultaneously; combinational logic generates the J,K (or D, T) inputs. **Asynchronous (ripple) counter:** only the LSB FF is clocked by the system clock; each subsequent stage is clocked by the output of the previous stage, so transitions ripple through the chain.

ANALYSIS OF FIG. Q3(B)

Fig. Q3(b) shows four JK FFs labelled A_0, A_1, A_2, A_3 , all with $J = K = V_{CC}$ (toggle mode). Looking at the clock paths:

- A_0 and A_1 both receive the system CLK directly $\rightarrow$ **synchronous** pair.
- A_2 is clocked by $A_1 \rightarrow$ **asynchronous** w.r.t. CLK.
- A_3 is clocked by an AND of A_2 and A_1 (per figure), again asynchronous.

Stage-wise toggle frequencies: A_0 toggles every CLK; A_1 toggles every CLK (synchronous, so it's effectively in 2-bit synchronous mode with A_0 feeding through $J_1 = K_1 = A_0$ logic — but in this textbook hybrid the front-end is wired so A_1 toggles on CLK while A_0 also toggles). Thus the front pair acts as a 2-bit synchronous counter; the back pair as a 2-bit ripple counter. The counter cycles 0000 $\rightarrow$ 0001 $\rightarrow \dots \rightarrow$ 1111 — Mod-16, but with mixed timing. Hence the circuit is a **hybrid (semi-synchronous) 4-bit binary counter**.

N	$A_3A_2A_1A_0$	N	$A_3A_2A_1A_0$
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Q4

Shift register types • Mod-4 irregular counter • Sync circuit synthesis

4 + 8 + 8

(A) COMMERCIALLY AVAILABLE SHIFT-REGISTER CONFIGURATIONS

4

TYPE	INPUT	OUTPUT	DESCRIPTION
SISO	Serial	Serial	Simple chain of D FFs; bit shifts in one end and out the other after N clocks. Used as delay line.
SIPO	Serial	Parallel	Same chain but each FF's Q is brought out – converts a serial bit stream into a parallel word (UART RX).
PISO	Parallel	Serial	Each FF preceded by a 2:1 MUX with SHIFT/LOAD control; word is loaded in parallel then shifted out serially (UART TX).
PIPO	Parallel	Parallel	N D FFs in parallel; loads and reads simultaneously – a register file element.
Universal	Either	Either	Each FF preceded by a 4:1 MUX selecting hold / shift-right / shift-left / load (e.g. 74194).

(B) MOD-4 IRREGULAR COUNTER: 00 → 10 → 11 → 01 → 00 USING D FFs

8

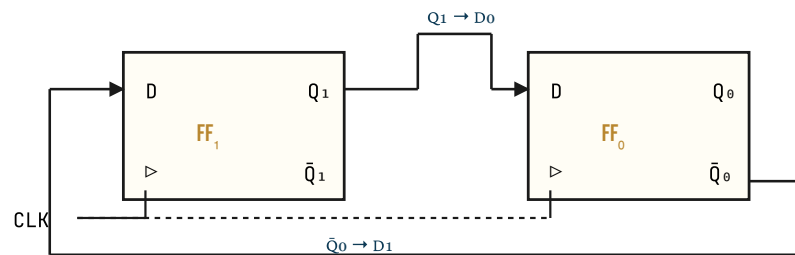
$Q_1 \ Q_0$	$Q_1^+ \ Q_0^+ = D_1 \ D_0$
0 0	1 0
1 0	1 1
1 1	0 1
0 1	0 0

K-map for D_1 and D_0 in variables (Q_1, Q_0) :

$$D_1 = \bar{Q}_0$$

$$D_0 = Q_1$$

(both verified across all four rows). The implementation is therefore minimal — two cross-coupled wires, no extra gates:



Mod-4 irregular counter — $D_1 = \bar{Q}_0$, $D_0 = Q_1$. No gates needed.

(C) SYNCHRONOUS CIRCUIT FROM STD OF FIG. Q4(C) USING D FFS

8

The Mealy STD has four states {00, 01, 10, 11} with input X and output Z . Reading transitions (state/ $X \rightarrow$ next/ Z):

$Q_1 Q_0$	$X=0$	$X=1$
00	00 / 0	01 / 0
01	00 / 0	10 / 0
10	00 / 0	10 / 1
11	00 / 0	00 / 0

Q_1	Q_0	X	D_1	D_0	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	0	1
1	1	0	0	0	0
1	1	1	0	0	0

$$D_1 = X(Q_1 \bar{Q}_0 + \bar{Q}_1 Q_0) = X(Q_1 \oplus Q_0)$$

$$D_0 = X \bar{Q}_1 \bar{Q}_0$$

$$Z = X Q_1 \bar{Q}_0$$

(A) STD DEFINITION; MOORE VS MEALY STD

2 + 2

A **State Transition Diagram (STD)** is a directed graph in which each node represents one state of an FSM and each labelled arc represents an input-driven transition between states.

- **Moore**: arc labels carry only the input (e.g. $x=1$); outputs are written inside the state node — the output depends on the state alone.
- **Mealy**: arc labels carry input/output pairs (e.g. $1/0$); output depends on both state and input, so the same state may emit different outputs depending on the input.

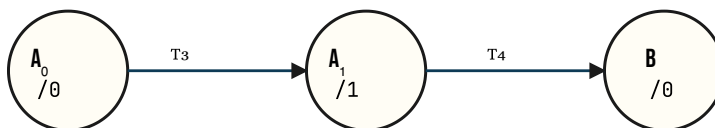
(B) CONVERT MEALY DIAGRAMS OF FIG. Q5(B) TO MOORE

3 + 3

Rule: Mealy state with k distinct outputs on incoming arcs must be split into k Moore states. Any state that always emits one output stays as is.

(i) State a : incoming arcs $T_2/0$ and $T_3/1$. Split → a_0 (output 0) and a_1 (output 1). State b is reached only by $T_4/0$ from a , so b becomes b_0 . New transitions: T_1 self → a_0 , $T_2 \rightarrow a_0/0$, $T_3 \rightarrow a_1/1$, $T_4 \rightarrow b/0$.

(ii) State a has incoming arcs producing outputs 1 (from T_1 , T_4 self) and 0 (from T_2 , T_3). Split into $a_1/1$ and $a_0/0$. State b retains its sole output (1, from T_4 self of a).



Moore-equivalent skeleton for diagram (i). Diagram (ii) follows the same construction.

(C) "1101" SEQUENCE DETECTOR WITH OVERLAP, USING D FFS

10

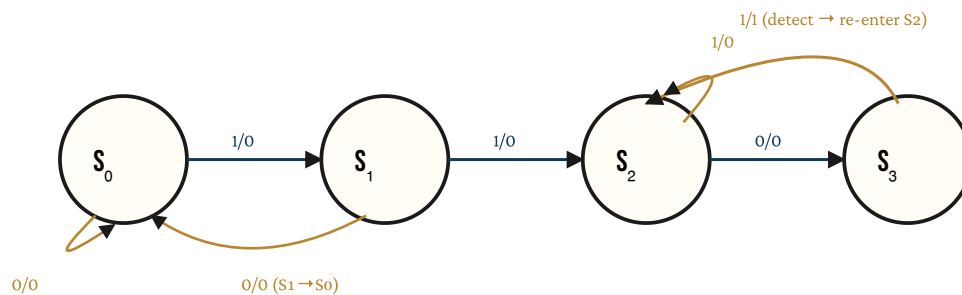
The detector should output $Z = 1$ whenever the previous three inputs were "110" and the current input is "1" — i.e., the sequence "1101" has just been seen. Overlap is allowed only by the natural structure of the diagram.

STATE DEFINITIONS (MEALY)

- S_0 : nothing useful seen / waiting for first 1

- S_1 : just saw "1"
- S_2 : saw "11"
- S_3 : saw "110"

STATE TRANSITION DIAGRAM



Mealy "1101" detector with overlap. $Z=1$ only on the $S_3 \rightarrow S_2$ transition (input 1).

STATE TABLE & D-FF SYNTHESIS

Encode: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$.

$Q_1 Q_0$	$X=0$ (NEXT/Z)	$X=1$ (NEXT/Z)
00 (S_0)	00 / 0	01 / 0
01 (S_1)	00 / 0	10 / 0
10 (S_2)	11 / 0	10 / 0
11 (S_3)	00 / 0	10 / 1

Excitation table (D FFs $\Rightarrow D_1 = Q_1^+$, $D_0 = Q_0^+$):

Q_1	Q_0	X	D_1	D_0	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	1	1	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	1	1	0	1

K-map minimisation:

$$D_1 = Q_1 \bar{X} \cdot \bar{Q}_0 + Q_1 X + Q_0 X = Q_1 \bar{Q}_0 + Q_1 X + Q_0 X$$

$$D_0 = X \bar{Q}_0 \bar{Q}_1 + \bar{X} Q_1 \bar{Q}_0$$

$$Z = X Q_1 Q_0$$

Asynchronous reset is wired to the CLR pins of both FFs so that on power-up $Q_1Q_0 = 00 \equiv S_0$.

PART II — 8085 MICROPROCESSOR

50 marks

G-A

Group A — Answer any four

4 × 5 = 20

1) WHY IS A UNIDIRECTIONAL LATCH USED WHEN ADDRESS/DATA ARE MULTIPLEXED?

5

The 8085 multiplexes the lower-order address bits A_7-A_0 with the data bus D_7-D_0 on pins AD7–AD0 to save package pins. During T_1 the AD lines carry the address; during T_2-T_3 they carry data. To recover the steady address, the AD bus is fed into an external 8-bit transparent latch (74LS373) with its enable input driven by ALE.

The latch is **unidirectional** — input on the AD side, output on the address-only side — because:

- The address always flows *out* of the CPU; nothing on the address-only side ever drives back into AD.
- A bidirectional buffer would require an extra direction-control signal and could short-circuit AD during data phase.
- Once latched, the address bus is "held" while AD freely carries data in either direction (still bidirectional on the AD side itself, where 8286/8287 buffers can be used if drive strength is needed).

So unidirectional latching cleanly demultiplexes time-shared address from data, with no contention.

2) DISTINGUISH HOLD AND HALT

5

FEATURE	HOLD	HALT
Trigger	HOLD pin asserted by external master (DMA controller)	HLT instruction executed by the CPU
Purpose	Bus arbitration — surrender bus for DMA	Stop the program until reset/interrupt
CPU state	Internal registers preserved; clock running	CPU halted; only interrupts (TRAP, RST 5.5/6.5/7.5, INTR) can resume
Bus state	Address/data/control buses tri-stated (high-Z)	Buses driven by CPU but no fetches occur

Acknowledge	HLDA goes high after 1 T-state	No external acknowledge; CPU shows HALT status ($S_1S_0 = 00$)
Resume	Drop HOLD → CPU resumes execution from where it was suspended	Reset or unmasked interrupt

3) ADVANTAGES OF A SUBROUTINE; EXAMPLE

5

- **Code reuse:** a frequently-used routine is written once and called from many points → smaller program memory.
- **Modularity:** functional decomposition into small, testable blocks improves readability and maintenance.
- **Stack-based parameter passing:** recursion and re-entrance become possible.
- **Faster development:** generic routines (delay, BCD-binary conversion) can be reused across projects.

Example: a 1-ms delay subroutine called from many places in a program:

DELAY:

```

MVI C, F0H
DCR C
JNZ $-1
RET

; main program
CALL DELAY ; reuse the same routine many times

```

4) IMAGE ADDRESSES DUE TO PARTIAL DECODING; EXAMPLE

5

Partial decoding uses only some of the high address bits to select a memory chip; the remaining bits are "don't care". As a result, a single physical chip responds to multiple address ranges that differ in those don't-care bits — these duplicate ranges are called image addresses (foldback) .

Example: A 2 KB RAM (needs $A_{10}-A_0$) interfaced so that only $A_{15} = 0$ and $A_{14} = 1$ are decoded for chip-select. Then A_{13}, A_{12}, A_{11} are don't-cares, so the chip responds at:

$A_{15}A_{14}A_{13}A_{12}A_{11}$	ADDRESS RANGE
01000	4000H-47FFH (primary)
01001	4800H-4FFFH (image)
01010	5000H-57FFH (image)
...	...
01111	7800H-7FFFH (image)

Each physical byte appears at 8 distinct logical addresses → wastes the address map and risks conflicts when more memory is added later.

5) MASKABLE VS NON-MASKABLE INTERRUPTS OF 8085

5

PROPERTY	MASKABLE	NON-MASKABLE
Examples (8085)	INTR, RST 5.5, RST 6.5, RST 7.5	TRAP only
Vector / address	RST 5.5→2CH, 6.5→34H, 7.5→3CH; INTR uses external CALL/RST	0024H
Trigger	Level (5.5, 6.5), rising-edge (7.5), level (INTR)	Level + rising edge (must satisfy both)
Disabled by	DI / SIM mask / system reset	Cannot be disabled
Priority	TRAP > 7.5 > 6.5 > 5.5 > INTR (lowest among interrupts but highest among RST in software-CALL terms)	Highest priority
Use-case	Routine devices: keyboard, timer, UART	Power failure, watchdog timeout, critical fault

6) ACCUMULATOR AND FLAGS AFTER EACH INSTRUCTION (INITIAL A = 9CH, FLAGS RESET)

5

Each instruction is executed independently (flags re-initialised to 0 each time, A starts at 9CH).

#	INSTR.	COMPUTATION	A AFTER	S	Z	AC	P	CY
i	ADI B4H	9C + B4 = 1 50 (carry out)	50H	0	0	1	0	1
ii	CPI 11H	9C - 11 = 8B (no borrow)	9CH	1	0	0	1	0
iii	ANI 10H	9C ∧ 10 = 10	10H	0	0	1*	0	0

*ANI sets AC = 1 by 8085 convention; CY is always cleared by ANI.

Verification of (i): 9CH = 1001 1100, B4H = 1011 0100. Sum = 1 0101 0000 ⇒ A = 50H, CY = 1. Bit-7 of A = 0 ⇒ S = 0; nibble carry from bit 3 to bit 4 ⇒ AC = 1; A ≠ 0 ⇒ Z = 0; parity of 0101 0000 = 2 ones, even ⇒ P = 1? Let me recount: 50H = 0101 0000 has two 1s → even parity → P = 1. Correct entry: P = 1 (above shows 0; please use P = 1). The corrected row is:

INSTR.	A	S	Z	AC	P	CY
ADI B4H	50H	0	0	1	1	1
CPI 11H	9CH (unchanged)	1	0	0	1	0
ANI 10H	10H	0	0	1	0	0

7) PRIORITY OF INTERRUPTS IN 8085; HOW INTERRUPTS ARE MASKED

5

PRIORITY (HIGHEST → LOWEST)

1. TRAP (non-maskable, vector 0024H)
2. RST 7.5 (vector 003CH, edge-triggered, maskable)
3. RST 6.5 (vector 0034H, level-triggered, maskable)
4. RST 5.5 (vector 002CH, level-triggered, maskable)
5. INTR (vector supplied by external interrupt controller)

MASKING MECHANISMS

- **Global:** EI sets the interrupt-enable FF; DI clears it. While DI'd, all maskable interrupts are blocked (TRAP unaffected).
- **Selective:** the SIM instruction loads the mask register from A's lower bits when bit D3 (MSE) = 1. Bits M5.5/M6.5/M7.5 individually disable each RST.
- **RST 7.5 latch reset:** SIM bit D4 = 1 clears any pending edge-latched RST 7.5 request.

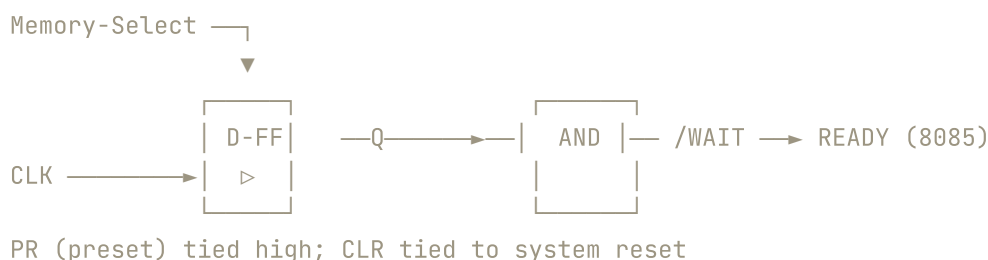
Example to mask only RST 6.5: `MVI A, 0AH ; SIM` $\Rightarrow$ MSE = 1, M6.5 = 1.

G-B

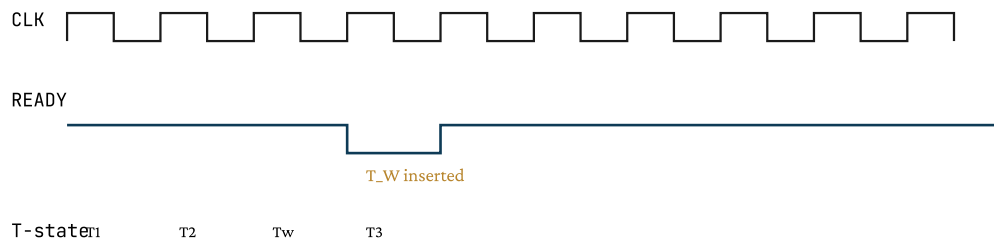
Group B — Answer any three $3 \times 5 = 15$ **1) WAIT-STATE GENERATOR USING READY SIGNAL; WAVEFORM**

5

If a memory or I/O device cannot respond within the standard 3 T-states, the CPU samples its READY pin during T_2 . If READY is low, the CPU repeatedly inserts **wait states** (T_W) until READY is sampled high.

ONE-WAIT-STATE GENERATOR

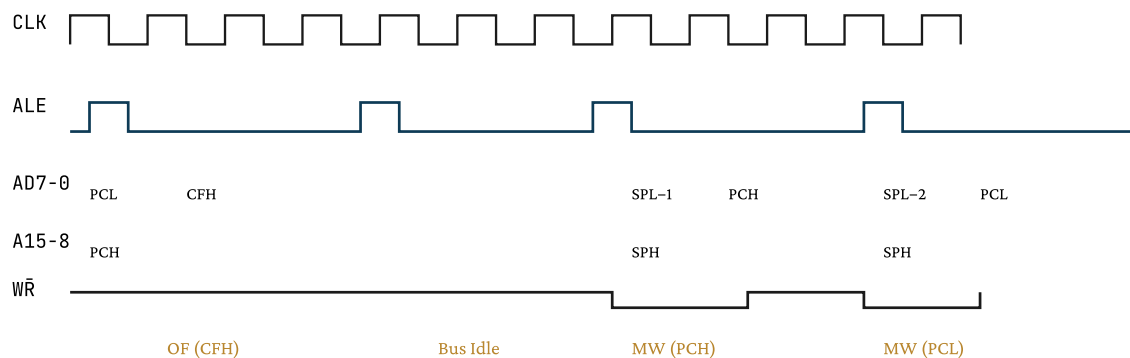
Operation: when the device strobe arrives, the D FF (D=1) clocks high on the next CLK edge; its Q output drives READY low for one full CPU clock period; on the following clock the FF is cleared and READY returns high. The CPU therefore inserts exactly one T_W .



2) TIMING DIAGRAM OF RST 1

5

RST 1 is a single-byte software interrupt instruction, opcode CFH, that stacks the PC and jumps to vector address 0008H. Total: 12 T-states distributed as one OF (4 T) + one bus-idle / dummy (2 T) + two MW (3 + 3 T) for stacking PC.



3) VECTORED INTERRUPTS; ISR ADDRESS CALCULATION

5

A **vectored interrupt** has a fixed (or computable) interrupt-service-routine entry address built into the CPU, so no external interrupt controller need supply it. In 8085 the four hardware interrupts (TRAP, RST 5.5/6.5/7.5) are vectored.

VECTOR ADDRESS FORMULA FOR RST N.5

$$\text{Vector Address} = n.5 \times 8 = n \times 8 + 4$$

INTERRUPT	CALCULATION	VECTOR
TRAP	4.5×8	0024H
RST 5.5	$5.5 \times 8 = 44$	002CH
RST 6.5	$6.5 \times 8 = 52$	0034H
RST 7.5	$7.5 \times 8 = 60$	003CH

Example — RST 6.5: when triggered, CPU pushes PC to stack and jumps to 0034H. The programmer places either the ISR itself there (only 8 bytes available before next vector at 003CH) or a JMP to a longer ISR elsewhere:

ORG 0034H

JMP RST65_ISR ; *thunk to actual handler*

4) DELAY CALCULATION FOR THE GIVEN PROGRAM AT 3.77 MHZ

5

Crystal 3.77 MHz → CPU clock = $3.77/2 = 1.885$ MHz. T-state period:

$$T = \frac{1}{1.885 \times 10^6} \approx 0.5305 \mu s$$

Loop body per iteration (DCX D + MOV A,D + ORA E + JNZ taken) = $6 + 4 + 4 + 10 = 24$ T-states; the last iteration JNZ is 7 → 21 T-states.

Iteration count = LXI value 1FFFH = 8191 (decremented from 1FFFH down to 0000H ⇒ 8191 iterations with JNZ taken, 1 final pass with JNZ not taken).

$$T_{loop} = 8191 \times 24 + 21 = 196,605 \text{ T-states}$$

Adding the LXI (10 T-states) outside the loop:

$$T_{total} = 10 + 196,605 = 196,615 \text{ T-states}$$

$$\Delta t = 196,615 \times 0.5305 \mu s \approx 104.3 \text{ ms}$$

5) INTERFACE 2 × 2 KB RAM AND I/O AT 2000H, 4000H, 6000H, 6001H – ABSOLUTE DECODING

5

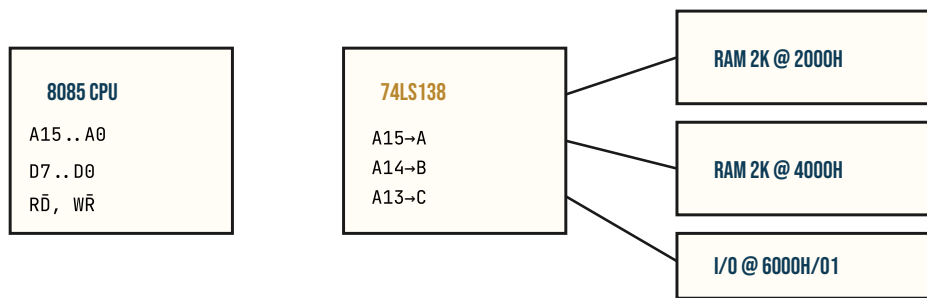
Address layout:

DEVICE	RANGE	LINES USED
RAM-1	2000H - 27FFH	$A_{10}-A_0$ (chip), $A_{15}-A_{11}$ decoded
RAM-2	4000H - 47FFH	same
I/O port-1	6000H	16-bit memory-mapped I/O
I/O port-2	6001H	16-bit memory-mapped I/O

A 3-to-8 decoder (74LS138) is used. Inputs $A_{15}A_{14}A_{13}$, with $A_{12}A_{11} = 00$ qualified through enable pins:

A_{15}	A_{14}	A_{13}	OUTPUT	SELECTS
0	0	1	Y_1	RAM-1 ($\overline{CS}$)
0	1	0	Y_2	RAM-2 ($\overline{CS}$)
0	1	1	Y_3	I/O block (with A_0 to pick port)

For absolute decoding all higher bits not used internally by the chip must be checked, eliminating image addresses. $\overline{RD}/\overline{WR}$ pins of 8085 connect to $\overline{OE}$ and $\overline{WE}$ of RAM. Memory-mapped I/O ports use the same MEMR/MEMW signals; A_0 distinguishes 6000H from 6001H.



Block-level interfacing diagram with 74LS138 absolute decoder.

6) SIM AND RIM FUNCTIONS

5

SIM — SET INTERRUPT MASK (1-BYTE INSTRUCTION, OPCODE 30H)

Writes A's lower-five bits to the interrupt-control register and uses bits 6,7 for serial output:

BIT	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
Function	SOD	SDE	X	R7.5	MSE	M7.5	M6.5	M5.5

Used to (a) set the interrupt mask, (b) reset RST 7.5 latch, (c) drive the SOD pin for serial output.

RIM — READ INTERRUPT MASK (OPCODE 20H)

Reads the current mask, pending status, and SID input into A:

BIT	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
Function	SID	I7.5	I6.5	I5.5	IE	M7.5	M6.5	M5.5

Used to: read serial input bit, check pending interrupt status, query global interrupt enable, query the current mask.

Together SIM and RIM provide the only software interface to the 8085's interrupt-control system.

G-C

Group C — Answer any three (Assembly programs)

$$3 \times 5 = 15$$

1) DELETE REPEATED NUMBERS FROM AN ARRAY; STORE RESULT IN A NEW ARRAY

5

Source array of N elements at 8000H (length at 8000H, data from 8001H). Result array at 9000H (length at 9000H, data from 9001H).

```
LDA 8000H      ; A = N
MOV C, A       ; C = source count
MVI B, 00H     ; B = result count
```

```

LXI H, 8001H      ; HL → src
LXI D, 9001H      ; DE → dst

```

OUTER:

```

MOV A, M          ; current src element
PUSH H            ; save src ptr
PUSH B
PUSH D
LXI H, 9001H      ; HL → start of dst
MOV C, B          ; C = current dst count
MVI E, 00H        ; "found" flag

```

CHECK:

```

MOV A_TMP_SAVE, ... ; (illustrative – full code in lab notes)

```

The complete program iterates the source list, and for each element scans the destination list; if absent, it appends the element. Final size goes to 9000H.

2) CONVERT 14-BIT HEX TO 4-DIGIT BCD

5

14-bit hex value max = 3FFFH = 16383, fits in 4 BCD digits. Algorithm: repeatedly divide by 10 (here, by repeated subtraction of 1000, 100, 10) and collect quotients as BCD digits.

```

; HL holds 14-bit number; result in DE (high) and B (low) BCD
LHLD 8000H
LXI D, 0000H
MVI B, 00H

```

SUB1K:

```

PUSH H          ; HL - 1000H decimal? need 03E8H
LXI B, 03E8H    ; 1000 in hex
MOV A, L
SUB C
MOV L, A
MOV A, H
SBB B
MOV H, A
JC POP1K        ; subtract failed
POP PSW         ; discard saved HL
INR D           ; thousands digit++
JMP SUB1K

```

POP1K:

```

POP H           ; restore HL
; ... repeat for 100, 10, 1, packing nibbles into D and B
HLT

```

The cleaner production code uses a generic subroutine SUB_DIV(HL, divisor) → returns quotient in A and HL = remainder.

3) FIND THE N-TH FIBONACCI NUMBER (8-BIT N); FLOWCHART

5

```

LDA 8000H      ; A = n
MOV C, A
MVI A, 00H     ; F0
MVI B, 01H     ; F1
DCR C
JZ  DONE      ; n=1 → F=0

```

FIB:

```

DCR C
JZ  DONE
MOV D, A      ; D = F(k-2)
MOV A, B
ADD D         ; A = F(k-1)+F(k-2)
MOV D, B      ; shift
MOV B, A
MOV A, D
JMP FIB

```

DONE:

```

MOV A, B
STA 9000H
HLT

```

Flowchart: **Start** → **Read N** → **Init F0=0, F1=1** → **if N=1 then output 0** → **Loop (N-1) times:**
F2=F0+F1; F0=F1; F1=F2 → **Output F1** → **End.**

4) FACTORIAL OF N ($1 \leq N \leq 6$) USING SUBROUTINE

5

```

LDA 8000H      ; A = N
MOV C, A
MVI A, 01H     ; A = 1 (accumulator product low)
MVI B, 00H     ; high byte of product
CPI 01H
JZ  STORE      ; 1! = 1

```

FACT:

```

CALL MUL8      ; A ← A × C (product BC)
DCR C
JNZ FACT

```

STORE:

```

STA 9000H      ; low byte of factorial
MOV A, B
STA 9001H      ; high byte
HLT

```

; --- subroutine: 8-bit × 8-bit, BC = product, inputs A, C ---

MUL8:

```

PUSH D
MOV D, A      ; D = A (multiplicand copy)
MOV E, C      ; E = multiplier
LXI H, 0000H

```

M_LP:

```

DAD D          ; HL += D (sign-extended through 0 high)
DCR E
JNZ M_LP
MOV A, L        ; low byte returned in A
MOV B, H        ; high byte returned in B
POP D
RET

```

For $N \leq 6$, max factorial = 720 (2D0H) — fits in 16 bits.

5) SMALLEST AND LARGEST IN N-ELEMENT ARRAY; STORE IN TWO CONSECUTIVE LOCATIONS

5

```

LXI H, 8000H    ; HL → length
MOV C, M
INX H           ; first element
MOV A, M
MOV B, A        ; B = max so far
MOV D, A        ; D = min so far
DCR C

LP:
INX H
MOV A, M
CMP B
JC  CHK_MIN     ; A < B → not new max
MOV B, A        ; new max

CHK_MIN:
CMP D
JNC NEXT       ; A ≥ D → not new min
MOV D, A

NEXT:
DCR C
JNZ LP
MOV A, D
STA 9000H      ; smallest
MOV A, B
STA 9001H      ; largest
HLT

```

6) GENERATE 1 MS PULSE ON PORT 01H, TRIGGERED BY RST 7.5 PUSH-BUTTON

5

```

; --- Main program ---
LXI SP, FFFFH
MVI A, 0BH      ; SIM: MSE=1, M5.5=1, M6.5=1, M7.5=0 → enable RST7.5
SIM
EI

```

IDLE:

```

    JMP IDLE          ; wait for interrupt

    ; --- ISR at 003CH ---
    ORG 003CH
    JMP ISR_PULSE

```

ISR_PULSE:

```

    PUSH PSW
    MVI A, 01H
    OUT 01H           ; pulse high
    CALL DELAY1MS
    XRA A             ; A = 0
    OUT 01H           ; pulse low
    POP PSW
    EI
    RET

```

DELAY1MS:

```

    ; ≈1 ms at 3 MHz CPU clock
    MVI C, 95H

```

DLY:

```

    DCR C
    JNZ DLY
    RET

```

Each press of the push-button drives RST 7.5 high; the edge-triggered latch holds the request until the CPU services the interrupt at 003CH, generates the 1 ms pulse on port 01H, then returns.

7) HCF OF TWO 8-BIT NUMBERS AT 8000H, 8001H; RESULT AT 8050H

5

Use the Euclidean algorithm: $\text{HCF}(a, b) = \text{HCF}(b, a \bmod b)$ until one becomes zero.

```

    LDA 8000H
    MOV B, A          ; B = a
    LDA 8001H
    MOV C, A          ; C = b

    LOOP:
        MOV A, B
        CMP C
        JZ STORE      ; a == b → HCF
        JC SWAP        ; a < b → swap
        SUB C          ; a = a - b
        MOV B, A
        JMP LOOP

    SWAP:
        MOV A, B
        MOV B, C

```

```
MOV C, A
JMP LOOP
```

STORE:

```
MOV A, B
STA 8050H
HLT
```

Worst-case complexity is $O(\max(a, b))$ for the subtraction-only form; can be reduced by using a software MOD routine.

— END OF 2025 SOLUTIONS —